EXAM DUTY MANAGEMENT SYSTEM

Deployment • System Architecture • Technical Report

Prepared By: Chandana IT Solutions	**Prepared For:** Chandana Sub Group / Operations
Date: October 2026	**Version:** 2.2 (Enterprise Handover Edition)
Repository: github.com/mayurChandanaitso/trackClockExam	**Platform Stack:** React 18 (Vite), NestJS 11, TypeORM, PostgreSQL/MySQL

1. Overview

The **Exam Duty Management System (TrackClock)** is a centralized web and mobile platform designed for assigning, monitoring, and validating examination personnel across multi-city testing networks. It replaces manual paperwork with a digital, conflict-free scheduling engine, digital attendance photo-proof verification, and dynamic month-wise analytical reporting.

The system comprises two core components:

- **Frontend Web Application:** A responsive React 18 (Vite) client providing an *Admin Management Portal* for operations managers and a mobile-optimized *Staff Web Portal* for invigilators and center staff.
- **Backend REST API:** A NestJS 11 (Node.js 22) application gateway providing role-based security, conflict-free scheduling algorithms, synchronous Multer attendance proof ingestion, and normalized relational persistence.

2. Architecture

Users interact with the frontend through any standard web or mobile browser. The client communicates via encrypted HTTPS REST requests to the NestJS backend API gateway, which interacts with the relational database through TypeORM with full foreign key constraints and connection pooling.

Layer	Technology	Hosting Specification
Frontend	React 18, TypeScript, Vite 6	Node.js Web App / Managed Cloud CDN
Backend API	NestJS 11, TypeORM 0.3, Node.js 22	Node.js Web App Gateway with PM2
Database	PostgreSQL / MySQL (Managed)	Managed Relational Cloud Database (8 Normalized Tables)

Layer	Technology	Hosting Specification
File Storage	Local Encrypted Storage / Cloud Bucket	Uploaded attendance proof images and PDF room documents
SSL / HTTPS	TLS 1.3 Encryption	Automated SSL Certificates

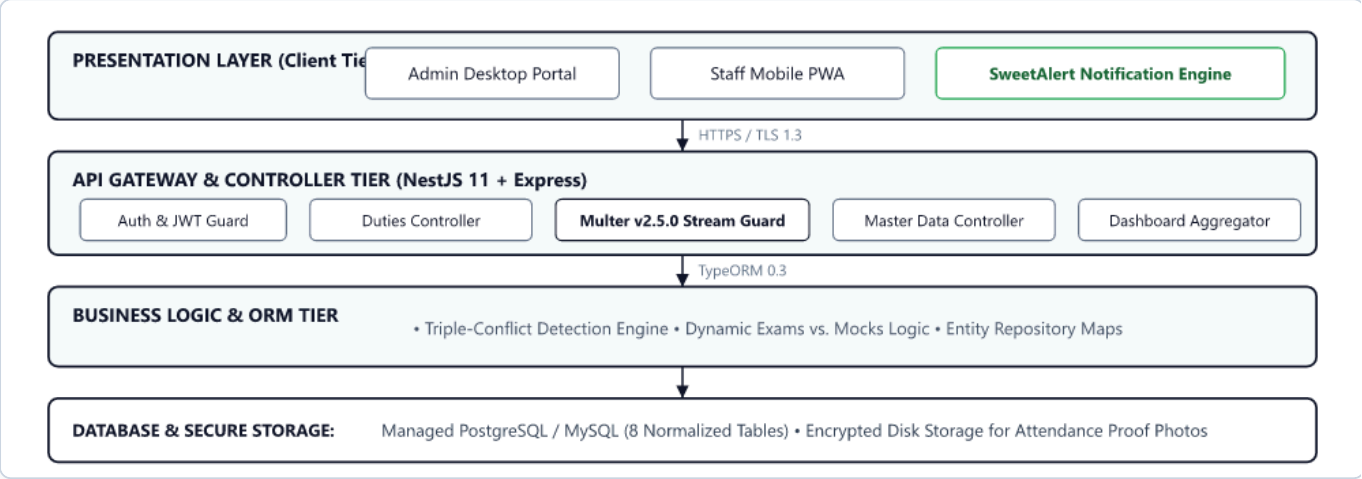


Figure 2.1: Multi-Tier System Component Architecture

3. Production URLs

Component	Production URL / Endpoint
Frontend (Application Portal)	`https://trackclockexam.chandanaitsolutions.com`
Backend API Base	`https://api-trackclockexam.chandanaitsolutions.com/api`
Health Check Probe	`https://api-trackclockexam.chandanaitsolutions.com/api/health`

Active Health Check Verification: A healthy backend returns the standardized response:
`{"status":"ok","service":"exam-duty-management-api","database":"connected"}`

4. Hosting Setup

The application runs as optimized Node.js Web Apps deployed directly from the GitHub version control repository. Each push or release to the main branch is built and deployed automatically.

4.1 Backend Web App Configuration

Setting Name	Configured Value
Source Repository	GitHub repository (branch `main`)

Setting Name	Configured Value
Framework Preset	NestJS / Node.js
Node.js Runtime Version	22.x
Root Directory	ExamDutyManagement/backend
Package Manager	npm
Build Command	npm run build
Output Directory	dist
Application Entry File	dist/main.js

4.2 Frontend Web App Configuration

Setting Name	Configured Value
Source Repository	GitHub repository (branch main)
Framework Preset	Vite / React
Node.js Runtime Version	22.x
Root Directory	ExamDutyManagement/frontend
Package Manager	npm
Build Command	npm run build
Output Directory	dist
Entry File	index.html (SPA Rewrites enabled)

5. Environment Variables Configuration

Application configuration settings are isolated into environment variables configured on the hosting server. Credentials must never be committed to public repositories.

5.1 Backend Environment Variables

Variable	Purpose & Description	Example / Production Setting
PORT	API Gateway port	3000
DB_HOST	Database server address	localhost / Managed DB Host

Variable	Purpose & Description	Example / Production Setting
DB_PORT	Database connection port	5432 (PostgreSQL) / 3306 (MySQL)
DB_DATABASE / DB_NAME	Database instance name	examduty_production_db
DB_USERNAME / DB_USER	Database user account	examduty_user
DB_PASSWORD	Database authentication password	Cryptographic Secret (Rotated)
DB_SSL	Enforce TLS database traffic	true in production
DB_SYNCHRONIZE	Auto-create database tables	false in production (Migrations used)
JWT_SECRET	Signing key for JWT tokens	Cryptographic Secret
CORS_ORIGIN	Permitted frontend origin	https://trackclockexam.chandanaitsolutions.com

5.2 Frontend Environment Variables

Variable	Purpose & Description	Example / Production Setting
VITE_API_URL	Base backend API gateway endpoint	https://api-trackclockexam.chandanaitsolutions.com/api
VITE_APP_TITLE	Application page title	Exam Duty Management System

6. Database Architecture & Data Dictionary

The persistence tier consists of 8 normalized relational tables configured with strict foreign key integrity, cascade delete protection, and unique constraints:

Table Name	Primary Key	Foreign Keys	Core Attributes & Constraints
duties	id (UUID)	employeeId, centerId, cityId, examId, roleId, shiftId, attendanceFileId	dutyDate (VARCHAR), dutyType ('Exam'|'Mock'), status ('Pending'|'Approved'|'Rejected'), reportingTime, shiftEndTime
employees	id (UUID)	None	resourceId (Unique VARCHAR), name, mobile, email, aadhaarNumber, panNumber, city, isActive

Table Name	Primary Key	Foreign Keys	Core Attributes & Constraints
exams	id (UUID)	None	code, name, type ('Exam'|'Mock'), description
centers	id (UUID)	cityId → cities(id)	centerCode, centerName, address, contactPerson
cities	id (UUID)	None	name (Unique: Mysore, Bangalore, Hassan, Mandya, etc.), state
roles	id (UUID)	None	code (Invigilator, Center Supt, Tech Support), title
shifts	id (UUID)	None	name ('Shift 1', 'Shift 2'), startTime, endTime
attendance_files	id (UUID)	Referenced by duties	originalName, filePath, mimeType, size, createdAt

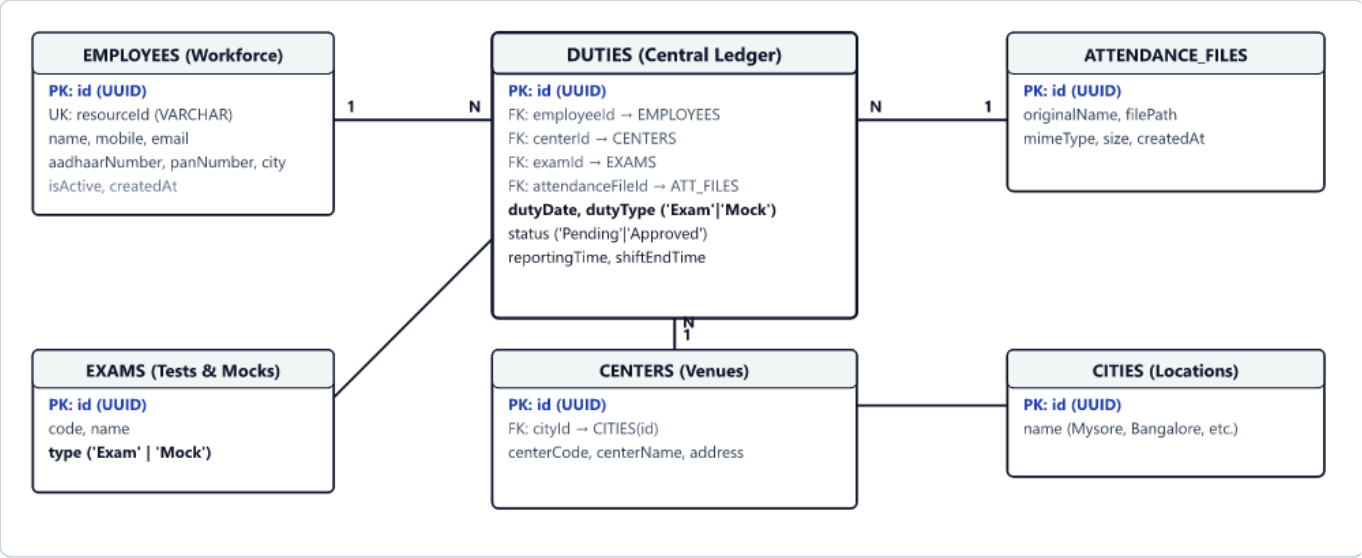


Figure 6.1: Entity-Relationship (ER) Relational Schema Model

7. End-to-End System Flow

The application coordinates workforce assignments across four deterministic stages:

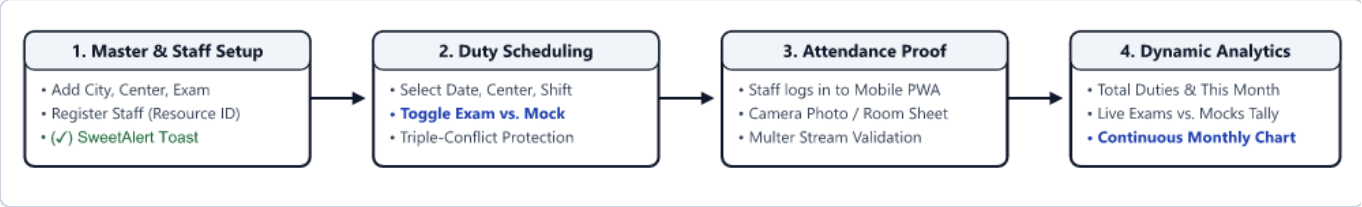


Figure 7.1: Continuous Operational System Flow

1. **Master Setup & Employee Onboarding:** Operations administrators populate baseline institutional data (cities, examination centers, exam rosters) and register personnel using verified Resource IDs. Real-time sequence validation prevents dummy test inputs. Upon saving, a floating SweetAlert toast confirms success.
2. **Duty Allocation & Conflict Engine:** Administrators assign duties selecting explicitly between **Exam** and **Mock** sessions. The scheduling engine validates employee availability, preventing duplicate bookings for the same personnel on matching dates and shifts.
3. **Staff Attendance Submission:** On testing day, field personnel sign into the mobile portal, review their assigned shifts under **Mark Duties**, and capture real-time photo proof or counter-signed room sheets. The Multer guard verifies file MIME types synchronously.
4. **Dynamic Real-Time Analytics:** System metrics aggregate instantly without page refreshes: Total Duties, This Month duties, Exam duties, and Mock duties, tracked continuously across a monthly histogram bar chart.

8. Detailed Portal Walkthrough & Functional Modules

The application architecture provides tailored user experiences for both administrators and field workforce members:

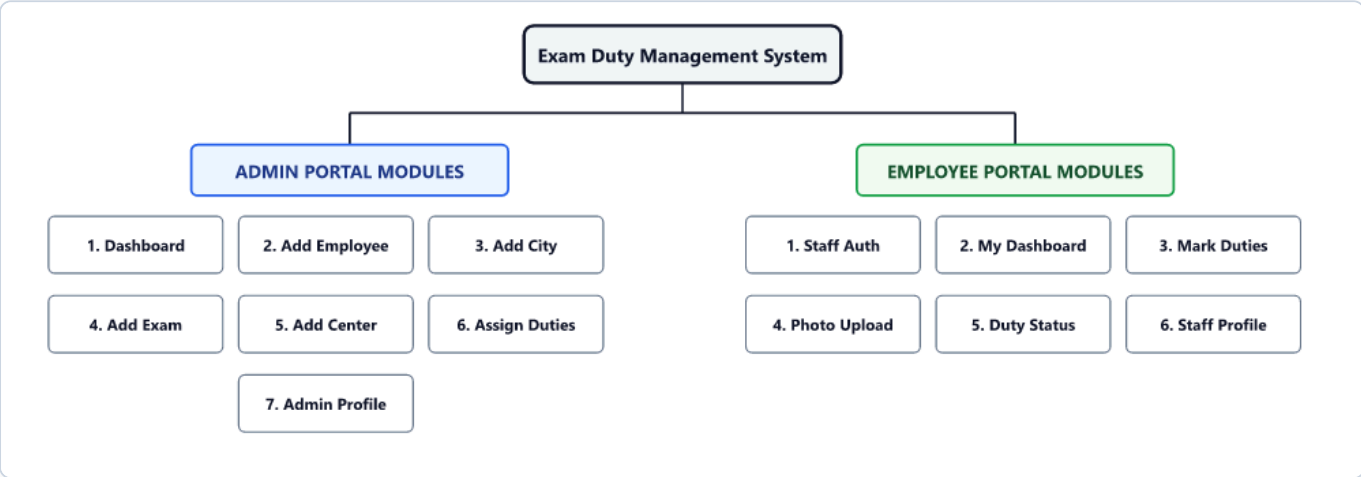


Figure 8.1: Portal Functional Architecture & Module Map

8.1 Admin Management Portal Modules

1. **Dashboard:** Executive monitoring center featuring 4 live KPI cards (*TOTAL DUTIES*, *THIS MONTH*, *EXAMS*, and *MOCKS*) alongside an interactive monthly duty overview bar chart (April through October) and the live **Mark Duties** feed.
2. **Add Employee (Workforce Management):** Single-entry staff registration enforcing Resource ID uniqueness, mobile verification, Aadhaar/PAN validation, and multi-city selection. Includes bulk Excel workforce ingestion and a paginated workforce directory. Clicking *Save & Register Employee* triggers the SweetAlert toast notification: `(✓) Employee "[Name]" registered successfully.`
3. **Add City (Location Master):** Management screen to register testing cities and administrative jurisdictions across Karnataka (Mysore, Bangalore, Hassan, Mandya, etc.).
4. **Add Exam Name (Examination Master):** Register examination sessions, defining exam code, descriptive title, and classifying whether the session is an official **Exam** or a preparatory **Mock Test**.
5. **Add Exam Center Name (Venue Master):** Register physical venues and testing institutions, linking venue address, center superintendent contacts, and geographic city assignment.
6. **Assign Duties to Employee (Duty Scheduling):** Allocation module allowing administrators to link an employee to an exam, date, center venue, role, and shift. Features a prominent toggle between **Exam** and **Mock** duty types and auto-populates reporting times. The conflict engine prevents overlapping assignments.
7. **Admin Profile:** Governance view managing administrative access credentials, session status, and security audit logs.

8.2 Employee / Staff Mobile Portal Modules

1. **Staff Authentication:** Streamlined authentication requiring only the employee's unique Resource ID and registered Mobile Number / PIN.
2. **Staff Dashboard:** Mobile-optimized overview displaying personal cumulative duties completed, current month quota, and assigned shifts.
3. **"Mark Duties" Feed:** Clean list of all assigned upcoming and current duties detailing exam name, duty date, center venue, assigned role, and shift timing.
4. **Attendance Proof Submission:** One-tap camera viewfinder integration allowing staff to capture live on-site attendance photos or upload counter-signed room invigilation sheets. Synchronous validation prevents process timeouts. Submitting attendance triggers the top-floating toast notification: `(✓) Attendance submitted successfully.`
5. **Duty Status Tracking:** Real-time status indicators (`Pending`, `Approved`, or `Rejected`) ensuring staff have continuous visibility into duty verification.
6. **Staff Profile:** Personal view displaying registered contact details, assigned operational cities, and KYC verification records.

9. Complete System & UML Diagrams

The following diagrams document the functional, behavioral, and structural architecture of the application:

9.1 UML Use Case Diagram

Illustrates system actors (Administrator, Staff Invigilator, Center Superintendent) and their interactions with core business use cases:

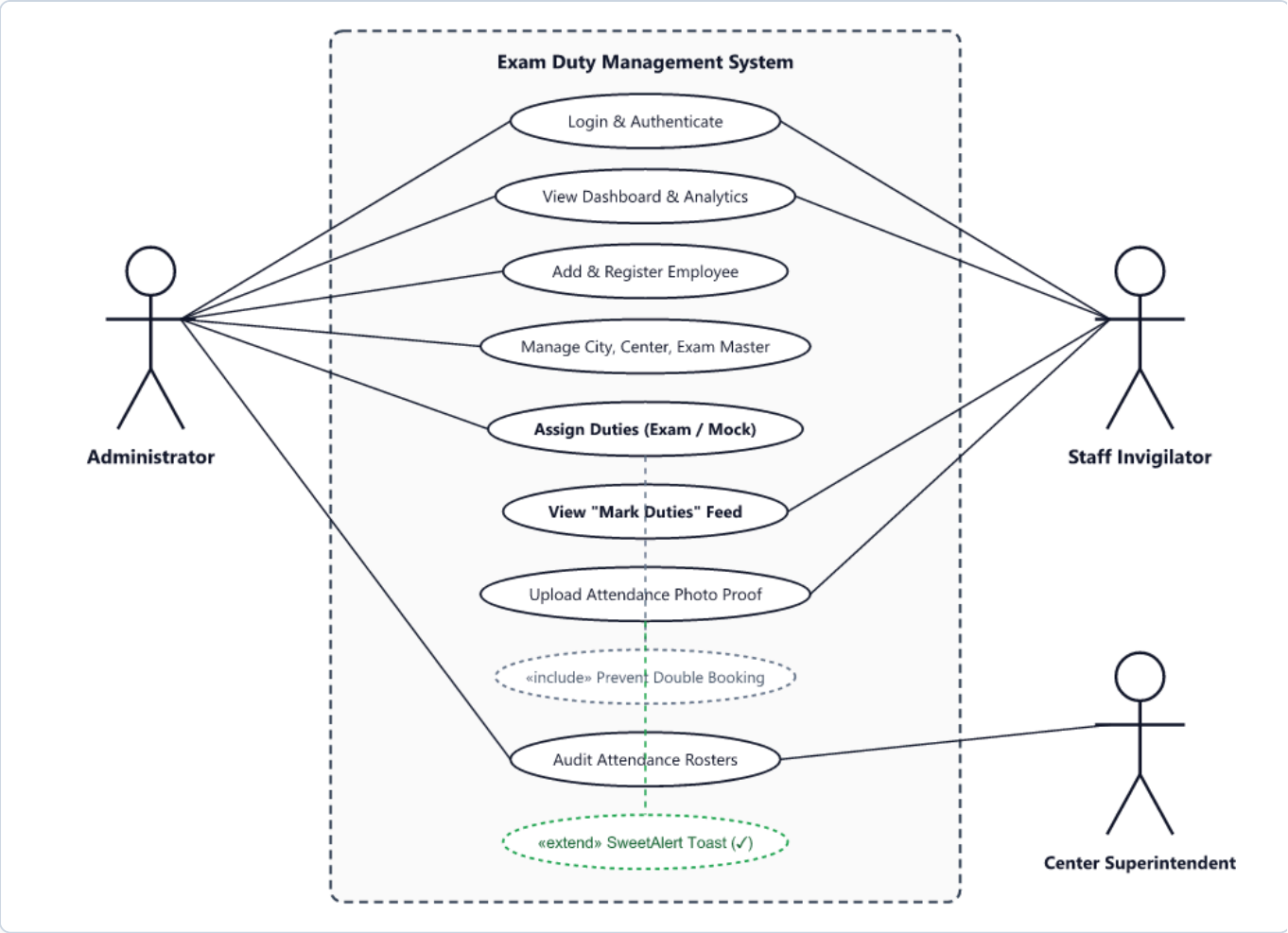


Figure 9.1: Formal UML Use Case Diagram (Standard Actor Notation)

9.2 Sequence Diagram (Attendance Submission Workflow)

Illustrates the chronological execution flow when an invigilator submits photo proof, passing through synchronous stream validation to database commit and user feedback:

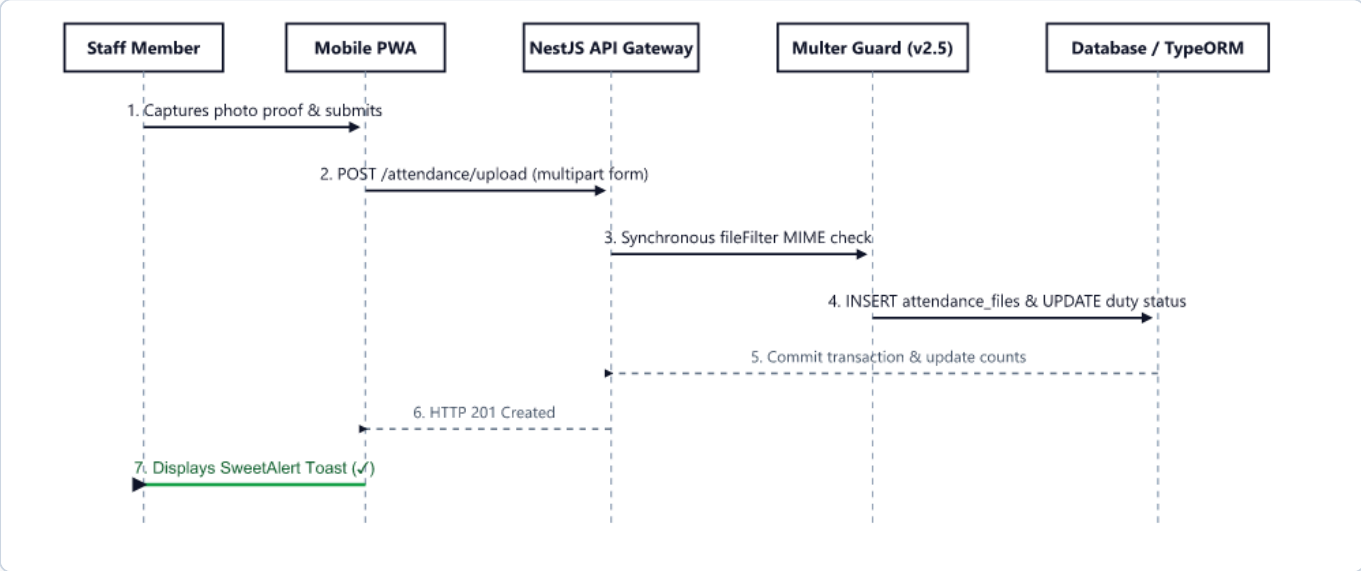


Figure 9.2: UML Sequence Diagram - Attendance Proof Submission

9.3 Duty Lifecycle State Machine Diagram

Models the progression of a duty assignment from initial administrative scheduling to final verification:

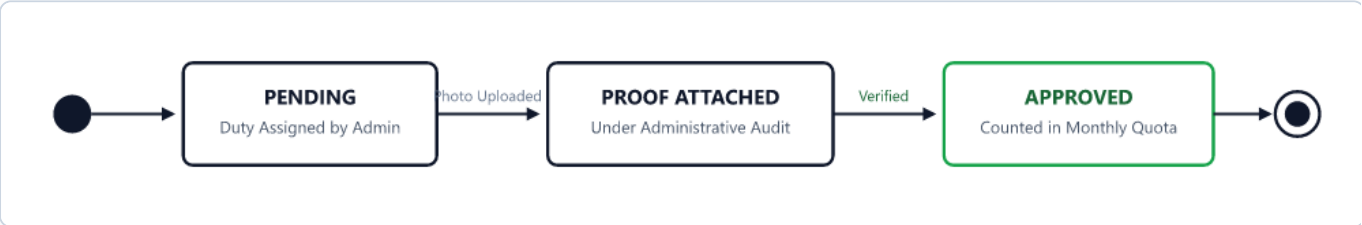


Figure 9.3: State Machine Diagram - Duty Lifecycle

9.4 Data Flow Diagrams (DFD Level 0 & Level 1)

Decomposes data flow across external entities, processing modules, and database data stores:

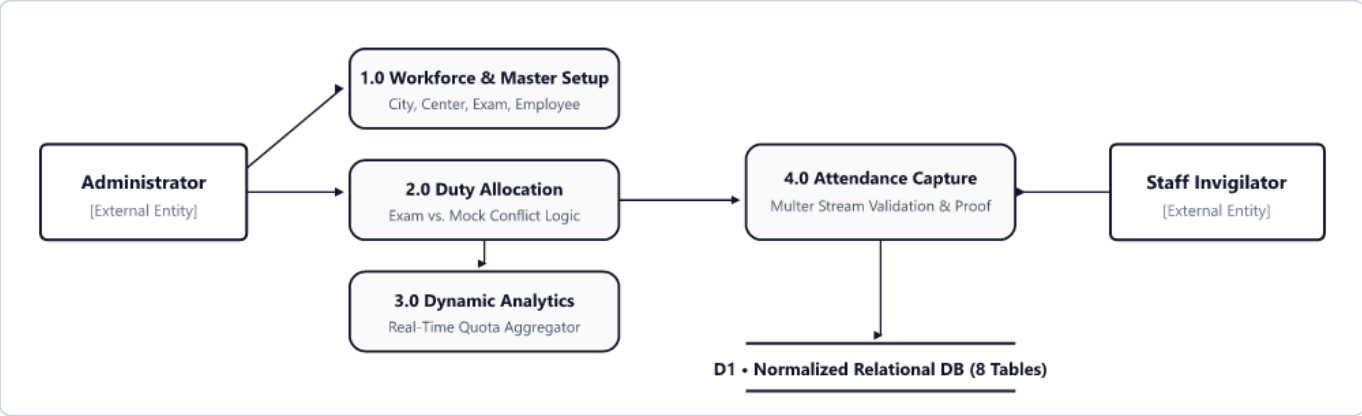


Figure 9.4: Data Flow Diagram (DFD Level 0 & Level 1 Decomposition)

10. Codebase Directory Structure & Configurations

The repository maintains a clean separation between backend micro-services, frontend views, and deployment configurations:

```
ExamDutyManagement/
├── backend/
│   ├── src/
│   │   ├── config/database.config.ts    # Database connection & connection pool settings
│   │   ├── entities/                    # 8 TypeORM entity schema models
│   │   ├── modules/
│   │   │   ├── attendance/              # Multer proof file stream handler
│   │   │   ├── auth/                    # Dual authentication & JWT guard
│   │   │   ├── dashboard/               # Metric aggregation & monthly calculations
│   │   │   ├── duties/                  # Duty CRUD & conflict detection engine
│   │   │   ├── master-data/             # Cities, Centers, Exams, Roles, Shifts
│   │   │   └── seed/                    # Baseline dataset seeder
│   │   ├── health/                      # Healthcheck probe controller
│   │   ├── app.module.ts                # Root application dependency container
│   │   └── main.ts                      # Bootstrap & CORS configuration
│   ├── package.json                     # Backend dependencies & build scripts
│   └── tsconfig.json                    # Backend TypeScript compiler configuration
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── MonthlyChart.tsx         # Monthly Exam vs. Mock bar chart
│   │   │   ├── RecentDutiesTable.tsx    # "Mark Duties" feed table component
│   │   │   ├── SearchableSelect.tsx     # Auto-complete select with dynamic font clamp
│   │   │   ├── StatCard.tsx             # 4-card metric KPI widget
│   │   │   └── SweetAlertToast.tsx      # Floating SweetAlert toast notification
│   │   ├── pages/                       # Screen views (AddDuty, AddEmployee, Dashboard)
│   │   ├── styles/App.css               # Corporate design tokens & styling
│   │   └── vite.config.ts               # Bundler settings & vendor chunk isolation
└── documentation/                       # Technical reports (.html, .doc, .docx, .pdf)
```

10.1 Key Configuration Files

- **`database.config.ts`:** Enforces connection pooling (`max: 20`, `idleTimeout: 30000ms`), SSL certificate validation, and disables destructive auto-synchronization in production.
- **`vite.config.ts`:** Implements Rollup manual chunking to separate large vendor libraries (React, React Router) and spreadsheet processing (XLSX) into distinct cached assets.
- **`tsconfig.json`:** Configured with strict type checking, decorator metadata enabled for TypeORM entities, and ES2022 target output.

11. Deployment Procedure (Summary)

1. **Subdomains:** Create the frontend and backend subdomains in the hosting control panel (e.g. `trackclockexam.chandanaitsolutions.com` and `api-trackclockexam.chandanaitsolutions.com`).
2. **Database Provisioning:** Create the relational database and user account in the hosting database manager, then import the database schema.
3. **Backend Web App Deployment:** Create a Node.js Web App linked to the GitHub repository using the settings in Section 4.1 and the environment variables in Section 5.1. Trigger deployment build.
4. **Verify Backend:** Open the health check URL (`/api/health`) and confirm `{"status":"ok","database":"connected"}`.
5. **Frontend Web App Deployment:** Create the frontend Web App using the settings in Section 4.2 and the build variable `VITE_API_URL`. Deploy.
6. **Verify Portal:** Access the application domain over HTTPS and verify administrative and staff authentication.

12. Updating the Application

1. Commit and push verified changes to the `main` branch on GitHub.
2. In the hosting panel, navigate to the relevant Web App (backend, frontend, or both) and trigger **Redeploy**.
3. Monitor the build log until deployment status indicates `Completed`.
4. Verify the health check endpoint and test portal login.

> **Fail-Safe Rollback:** If a deployment fails, the previous working version remains active automatically. Review the build log, resolve the error, and re-trigger deployment.

13. Access and Credentials Matrix

User Role	Portal Interface	Login Identifier	Default Password / PIN	Access Privileges
Super Administrator	Desktop Management Portal	`admin` or `17655`	`admin123`	Full governance: Staff KYC, duty scheduling (Exam/Mock), attendance audits, Excel exports.
Operations Manager	Desktop Management Portal	`mohan`	`admin123`	Center coordination, duty allocation, attendance sheet reviews.

User Role	Portal Interface	Login Identifier	Default Password / PIN	Access Privileges
Staff Invigilator	Mobile Web PWA	Resource ID (e.g. 23625)	Registered Mobile PIN	Mobile view: "Mark Duties" feed, photo proof submission, monthly quota tracking.

14. Security Hardening & Enhancements (v2.2)

- **Multer CVE-2025-47279 Remediation:** Upgraded to `multer@2.5.0` and enforced synchronous `fileFilter` functions across all upload routes. This eliminates stream abort unhandled errors and completely prevents backend process crashes.
- **SweetAlert Toast Notification Standard:** Replaced disruptive modal dialogs with clean, floating top-center toast notifications featuring an emerald checkmark badge ((✓)).
- **Anti-Sequence Input Protection:** Enforced client-side and server-side sequence checks that reject continuous dummy sequences (e.g. 123456) in mobile and Aadhaar fields.
- **Neutral Form Initialization:** Duty date, city, center, exam, role, shift, and employee are unselected by default, preventing unintentional duplicate submissions.

15. Backups and Maintenance

- **Database Backups:** Enable automated daily snapshots on the hosting database manager and export an offline SQL dump prior to major application version updates.
- **Uploaded Document Storage:** Attendance proof images and room sheets are stored in the persistent uploads directory. Ensure this directory is linked to persistent block storage to survive container redeployments.
- **Uptime Monitoring:** Configure automated uptime monitoring on the `/api/health` endpoint to notify administrators immediately in the event of database connectivity drops.

16. Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action
Health check shows database error	Wrong database name, host, or credentials	Verify environment variables in Section 5.1 and confirm database server is reachable.
Build fails with `command not found`	Missing dependencies or wrong node version	Ensure Node.js 22.x is selected and package dependencies are installed via `npm install`.

Symptom	Likely Cause	Corrective Action
Login displays network error	`VITE_API_URL` missing or incorrect at build time	Set the correct backend URL in frontend environment variables and redeploy the frontend.
Portal opens but displays empty tables	CORS origin misconfiguration	Confirm `CORS_ORIGIN` on backend includes the exact frontend domain without trailing slashes.
Page reload returns HTTP 404	SPA rewrite rule missing	Ensure single-page routing rewrite rules (`index.html`) are enabled in hosting configuration.

17. Operational Handover Checklist

- ☐ Frontend and backend subdomains operational and secured over HTTPS (TLS 1.3).
- ☐ Backend REST API returning `database: connected` on healthcheck probe.
- ☐ Super Admin, Operations Manager, and Staff logins verified.
- ☐ SweetAlert toast notifications operational across employee registration and attendance upload.
- ☐ Dynamic Exam vs. Mock counting active across summary cards and monthly histogram charts.
- ☐ Database credentials excluded from version control and secured on hosting server.
- ☐ Full project technical documentation delivered in HTML, Microsoft Word (.doc), and True PDF formats.

© 2026 Chandana IT Solutions. All Rights Reserved.